ZERO-KNOWLEDGE ARCHITECTURE

Raccoon Chat

Decentralized, Private Real-Time Chat with End-to-End Cryptography

Curve25519 / X25519 | AES-256-GCM | BIP39 Mnemonic | WebSockets

HogoBear Engineering Team

PROBLEM & VISION

Project Idea

Overcoming the fundamental security flaws of centralized messaging systems.

⚠️

The Centralized Problem

Traditional apps (WhatsApp, Telegram, Messenger) rely on central cloud databases that store chat records, message metadata, and user graphs.

- Servers vulnerable to government subpoenas and corporate breaches.
- Phone numbers required for identity, opening vectors for SIM-swapping.
- Cloud backups frequently store plaintext conversations.



The Raccoon Solution

Raccoon Chat implements a **Zero-Knowledge Relay model**: the server acts strictly as a real-time router, never retaining user messages.

- Decentralized identity generated via BIP39 12-word seed phrases.
- Diffie-Hellman (X25519) on-device shared secret calculation.
- Ghost Mode: messages exist purely in device volatile RAM memory.

CAPABILITIES

Core Features

Engineered for uncompromising privacy, speed, and cryptographic autonomy.

🔑

Mnemonic Identity

Account initialization using BIP39 12-word mnemonic phrases. No phone numbers, no email tracking.

⚡

On-Device Keypair

Automated Curve25519/X25519 key generation stored locally in device SecureStore.

🔒

AES-256-GCM

Authenticated symmetric encryption with unique 12-byte IVs for each dispatched message.

💬

Real-Time Sockets

Low-latency WebSocket rooms delivering instant message routing and delivery receipts (✓ / ✓✓).

👻

Ghost RAM Mode

Default ephemeral storage: chats live exclusively in Zustand RAM, deleted immediately on app exit.

👥

Token-Based Contacts

Add friends via cryptographic token IDs with friend request approvals and contact blocking.

SYSTEM DESIGN

Architecture Overview

A decoupled three-tier topology focused on stateless message routing.

MOBILE CLIENT

React Native App

- Expo SDK 57 & Expo Router
- Zustand in-memory state
- TweetNaCl & Node-Forge engines
- SecureStore & SQLite at rest

WSS / HTTPS

⇄

JWT Handshake

REAL-TIME RELAY

Node.js & Express 5

- Socket.io event-driven rooms
- Ephemeral packet forwarding
- Auth, Presence & Typing sockets
- Zod schema payload validation

Sequelize ORM

⇄

Port 5433

METADATA STORE

PostgreSQL 16

- Docker Compose orchestrated
- Persists Users, Friends, Rooms
- Stores Public Keys exclusively
- **0 Messages Saved**

🛡️ **Zero-Knowledge Guarantee:** All message payloads travel across the network fully encrypted. The relay server is cryptographically blind to plaintext content.

CLIENT INTERNALS

Mobile App Structure

Modular front-end engineering designed for performance and reactive state management.

📁 app/ (Expo Router)

- **File-Based Routing:** Clean route hierarchy (`app/(main)/chats/[conversationId].jsx`).
- **Dynamic Headers:** Real-time contact presence, avatar, and active status indicators.
- **Safe Views:** Optimized keyboard handling for smooth chat scrolling.

🧠 src/store/ (Zustand)

- **useAuthStore:** Stores user JWT, keys, and presence maps.
- **useMessageStore:** RAM-only store holding active conversation transcripts.
- **Immediate GC:** State wiped clean upon component unmount or app kill.

🔌 src/hooks/useChatSocket.js

- **Automatic Connection:** JWT auth handshake on mount.
- **Room Listeners:** `receive_message`, `message_sent_ack`, `user_typing`.
- **Action Emitters:** `sendMessage`, `startTyping`, `sendDeliveryAck`.

🔐 src/crypto/ (Security Engine)

- **keys.js:** X25519 scalar multiplication to derive 32-byte shared secret.
- **encrypt.js:** AES-256-GCM encryption returning `{ iv, ciphertext }`.
- **decrypt.js:** Authenticated decryption and UTF-8 decoding in RAM.

CRYPTOGRAPHIC PROTOCOL

How E2EE Encryption Works

The cryptographic handshake and message transmission pipeline.

1

Keypair Initialization

Alice and Bob generate independent Curve25519 / X25519 keypairs upon onboarding. Private keys never leave the phone.

2

Public Key Exchange

Upon opening a conversation, clients publish their public keys to each other through the WebSocket handshake.

3

Diffie-Hellman Shared Secret (ECDH)

Alice combines her Private Key with Bob's Public Key: `rawShared = nacl.scalarMult(alicePriv, bobPub)`. Both derive identical 32-byte AES keys.

4

Authenticated Message Encryption

Sender generates a cryptographically random 12-byte IV and encrypts plaintext with `AES-256-GCM`, appending an authentication tag.

5

Blind Server Relay & RAM Decryption

Server receives only `{ iv, ciphertext }` and forwards it. Recipient decrypts it in volatile RAM with zero disk footprints.

DATA LAYER

Database Models (PostgreSQL)

Minimalist server schema storing only authentication credentials and contact graphs.

Users	Model
🔑 id	UUID
username	STRING (UK)
tokenUser	STRING (UK)
publicKey	STRING
password	HASH
recovery_seed	HASH (UK)
is_online	BOOLEAN

Friends	Model
🔑 id	UUID
🔗 user_id	UUID
🔗 friend_id	UUID
is_blocked	BOOLEAN
status	ENUM
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

Conversation	Model
🔑 id	UUID
🔗 user_one	UUID
🔗 user_two	UUID
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

💡 **Crucial Privacy Highlight:** The database contains NO "Messages" table. Communication contents exist only in transit and on user devices.

DEFENSE IN DEPTH

API & Security Standards

Multilayered validation, transport security, and cryptographically verified sessions.



Zod Schema Validation

Every HTTP endpoint and WebSocket event payload is strictly validated at runtime using Zod schemas, blocking injection attacks and malformed payloads.



JWT Handshake Auth

WebSocket connections authenticate via JWT in the initial handshake. Sockets join private rooms mapped to verified user identities.



AES-256-GCM Integrity

Galois/Counter Mode (GCM) combines confidentiality with authentication: any message altered in transit fails decryption immediately.



Hardened Infrastructure

HTTP headers protected by Helmet, bcrypt salted password hashing (work factor 10), and strict CORS origin controls.

TECH STACK

Technology Stack

Modern, robust, and industry-standard tools across the entire stack.

TIER / DOMAIN	TECHNOLOGIES & LIBRARIES
Mobile Client	Expo SDK 57, React Native 0.86, React 19, Zustand 5, Expo Router
Backend Relay	Node.js 20, Express 5, Socket.io 4, Sequelize ORM 6
Database & Infra	PostgreSQL 16, Docker Compose, Alpine Linux Containers
Cryptography	TweetNaCl, Libsodium Wrappers, Node-Forge, @scure/bip39, Curve25519
Security & Auth	JSON Web Tokens (JWT), Bcrypt, Zod 4, Helmet 8, SecureStore

DEPLOYMENT

How to Run It

Three simple commands to spin up the entire ecosystem locally.

```
$ docker compose up -d # 1. Start PostgreSQL 16 database container
$ cd backend && npm run dev # 2. Launch Express & Socket.io server with nodemon
$ cd frontend && npx expo start # 3. Launch React Native Metro bundler & Expo Go
```

1. Database

Spins up PostgreSQL container on port 5433 with persistent volumes.

2. Relay Server

Initializes Sequelize models and registers real-time socket namespaces on port 3000.

3. Mobile App

Provides QR code for instant loading via Expo Go on iOS and Android.

CONCLUSION

Thank You

Any questions or live demonstration requests?

Raccoon Chat — Confidentiality by Architecture

Open Source • Zero-Knowledge • Peer-to-Peer Encryption